#1-project-overview
#2-environment-setup
#3-dataset-description
#4-architecture-overview
#5-module-reference
#6-data-pipeline
#7-feature-engineering
#8-model-training
#9-evaluation-framework
#10-interpretability
#11-confidence-scoring
#12-blind-prediction-pipeline
#13-results-summary
#14-data-leakage-prevention
#15-reproducibility
#16-extension-guide
#17-troubleshooting

```
    fair-esm
    Bio.SeqIOBio.SeqUtils.ProtParam
```

```
# 1. Clone the repository
git clone <repo-url> && cd protein-classification

# 2. Create and activate a virtual environment
python -m venv .venv && source .venv/bin/activate

# 3. Install PyTorch with CUDA support (adjust URL for your CUDA version)
pip install torch --index-url https://download.pytorch.org/whl/cu124

# 4. Install remaining dependencies
pip install -r requirements.txt
```

```
biopython>=1.83
numpy>=1.26
pandas>=2.1
scikit-learn>=1.4
xgboost>=2.0
lightgbm>=4.2
torch>=2.1
fair-esm>=2.0
matplotlib>=3.8
seaborn>=0.13
```

```
shap>=0.44
joblib>=1.3
scipy>=1.12
imbalanced-learn>=0.12
tqdm>=4.66
```

---

class0_rep_seq.fasta.txtec1_rep_seq.fasta.txtec2_rep_seq.fasta.txtec3_rep_seq
ec4_rep_seq.fasta.txtec5_rep_seq.fasta.txtec6_rep_seq.fasta.txt

---

```
┌──────────────────────────────────────────────────────────────────┐
│                          Data Layer                              │
│  data_loading.py: FASTA parsing → DataFrame → stratified CV splits │
└──────────────────────────────────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────────────────────────────────┐
│                       Feature Layer                              │
│  composition.py      → 421-d (AA freq + dipeptide + length)      │
│  physicochemical.py  → 8-d  (MW, pI, GRAVY, charge, SS fracs)    │
│  embeddings.py       → 320-d / 1280-d (ESM-2 mean-pooled)        │
│  Combined            → 1288-d (ESM-2 650M + physicochemical)     │
└──────────────────────────────────────────────────────────────────┘
              │
              │
              ▼
┌──────────────────────────────────────────────────────────────────┐
│                       Training Layer                              │
│  training.py: leak-safe CV loop (scaler + SMOTE on train only)    │
│  baseline.py:  Logistic Regression, Random Forest                 │
│  advanced.py:  XGBoost, LightGBM, ablation, threshold optimisation │
│  finetune.py:  ESM-2 8M end-to-end fine-tuning (PyTorch)          │
│  ensemble.py:  Soft-vote fusion of fine-tuned 8M + XGBoost 650M   │
└──────────────────────────────────────────────────────────────────┘
              │
```

```
            │
            ▼
┌──────────────────────────────────────────────────────────────────┐
│                    Evaluation & Output Layer                      │
│  evaluation.py:       metrics, confusion matrices, comparison plots │
│  interpretability.py: feature importance, SHAP, error analysis      │
│  confidence.py:       probability → High / Medium / Low calibration │
│  predict_blind.py:    FASTA → model → formatted predictions         │
│  generate_plots.py:   combined model comparison figures              │
└──────────────────────────────────────────────────────────────────┘
```

```
Phase 1: python -m src.data_loading          # verify data
Phase 2: python -m src.models.baseline       # handcrafted baselines
Phase 3: python -m src.features.embeddings   # ESM-2 extraction
Phase 4: python -m src.models.advanced       # XGBoost/LightGBM + ablation
Phase 5: python -m src.models.finetune       # end-to-end fine-tuning
Phase 6: python -m src.models.ensemble       # ensemble evaluation
Phase 7: python -m src.interpretability      # feature importance & SHAP
Phase 8: python -m src.confidence            # confidence calibration
```

---

src/data_loading.py

load_all_sequences(root: Path) → pd.DataFrameseq_idsequencelabellength
get_cv_splits(labels, n_splits=5, seed=42) → list[tuple]print_class_distribution
(df) → None

SEED = 42
FASTA_LABEL_MAP
CLASS_NAMES["Not enzyme", "Oxidoreductase", "Transferase", "Hydrolase",
"Lyase", "Isomerase", "Ligase"]

---

src/features/composition.py

amino_acid_composition(seq)(20,)dipeptide_frequencies(seq)(400,)extract_compo
(N, 421)get_feature_names()list[str]

---

src/features/physicochemical.py Bio.SeqUtils.ProtParam

ProtParam.molecular_weight()ProtParam.isoelectric_point()ProtParam.charge_at_

(N, 8)get_feature_names()

---

src/features/embeddings.py

esm2_t6_8M_UR50Desm2_t33_650M_UR50D

```
extract_esm2_embeddings(sequences, model_name, batch_size, device)

load_or_compute_embeddings(sequences, cache_path, model_name)
```

.npyoutputs/features/

---

src/training.py

```
cross_validate_model(
    model_fn,          # callable returning a fresh model instance
    X, y,              # features and labels
    cv_splits=None,    # pre-computed fold indices (or generated internally)
    n_splits=5,
    use_scaler=True,   # fit StandardScaler on train fold only
    use_smote=False,   # apply SMOTE on train fold only
    use_class_weight=False,
    model_name="model"
) -> dict
```

```
{
    "model_name": str,
    "fold_metrics": [{"accuracy": ..., "macro_f1": ..., ...}, ...],
    "summary": {"accuracy_mean": ..., "accuracy_std": ..., ...},
    "oof_preds": np.ndarray,   # out-of-fold predictions
    "oof_proba": np.ndarray,   # out-of-fold probability vectors
    "oof_true": np.ndarray     # ground truth labels
}
```

optimize_thresholds(y_true, y_proba)scipy.optimize.minimize

---

src/models/baseline.py

```
   class_weight='balanced'max_iter=1000solver='lbfgs'n_estimators=300max_depth=2
class_weight='balanced'PCA(50) → RandomForest
   cross_validate_model
```

---

src/models/advanced.py

```
   outputs/models/best_model.joblib

{
    "model": trained_xgboost,
    "scaler": fitted_StandardScaler,
    "feature_source": "ESM-2 + Physicochemical",
    "esm_model": "esm2_t33_650M_UR50D",
    "thresholds": np.ndarray,
    "oof_preds": np.ndarray,
    "oof_proba": np.ndarray,
    "oof_true": np.ndarray
}
```

---

src/models/finetune.py

```
ESM-2 8M (6 transformer layers, fully unfrozen)
    → Mean-pool over sequence tokens
    → Linear(320, 128)
    → GELU
    → Dropout(0.3)
    → Linear(128, 7)
```

batch_size × grad_accum

outputs/models/finetune_artifact.joblibFinetunePredictorpredict_proba

---

src/models/ensemble.py

p_ensemble = w_ft × p_finetune + w_xgb × p_xgboost

EnsemblePredictorpredict_proba(sequences)

---

src/evaluation.py

compute_metrics(y_true, y_pred){accuracy, macro_f1, balanced_accuracy, mcc}plot_confusion_matrix(...)print_metrics_table(results_list)plot_metrics_comp plot_class_distribution(...)plot_sequence_length_distribution(...)save_results_j path)

---

src/interpretability.py

TreeExplainer

---

src/confidence.py

confidence_calibration_report

outputs/figures/reliability_diagram.png

---

src/predict_blind.py

```
# Ensemble (recommended)
python -m src.predict_blind \
    --fasta <test.fasta> \
    --model outputs/models/best_model.joblib \
    --model-finetune outputs/models/finetune_artifact.joblib \
    --output outputs/predictions/blind_predictions.txt
```

```
SEQ01 1 Confidence High
SEQ02 0 Confidence Medium
SEQ03 3 Confidence Low
```

```
    Bio.SeqIO
```

---

src/generate_plots.py

```
   baseline_results.jsonadvanced_results.jsonfinetune_results.jsonensemble_resul
```

```
    outputs/figures/model_comparison.png
    outputs/figures/ablation_results.png
```

```
from pathlib import Path
from src.data_loading import load_all_sequences, get_cv_splits

# Load all 39 764 sequences
df = load_all_sequences(Path("."))
sequences = df["sequence"].tolist()
labels = df["label"].values
```

```
# Generate stratified 5-fold CV splits (seed = 42)
cv_splits = get_cv_splits(labels, n_splits=5, seed=42)

# Each fold preserves the original class proportions
for train_idx, val_idx in cv_splits:
    print(f"Train: {len(train_idx)}, Val: {len(val_idx)}")
```

```
seed = 42
class_weight='balanced'scale_pos_weight
```

```
Logistic Regression (handcrafted)     → Macro F1: 0.191  (floor)
Random Forest (handcrafted)           → Macro F1: 0.141
XGBoost (ESM-2 650M + physico)        → Macro F1: 0.575  (+200 %)
XGBoost + SMOTE                       → Macro F1: 0.595  (+3.5 %)
```

```
Fine-tuned ESM-2 8M                    → Macro F1: 0.509
Ensemble (8M FT + XGBoost 650M)        → Macro F1: 0.625  (+5 %)
```

```
from src.evaluation import compute_metrics

metrics = compute_metrics(y_true, y_pred)
# {'accuracy': 0.886, 'macro_f1': 0.595, 'balanced_accuracy': 0.552, 'mcc': 0.63
```

```
outputs/

    baseline_results.json
    advanced_results.json
    finetune_results.json
    ensemble_results.json
    confidence_results.json
    interpretability_results.json
```

```
molecular_weight
```

```
from src.confidence import assign_confidence

confidences = assign_confidence(proba)  # proba shape: (N, 7)
# Returns: ["High", "Medium", "Low", ...]
```

---

src/predict_blind.py

--model--model-finetune

```
# Single model
python -m src.predict_blind \
    --fasta blind_test.fasta \
    --model outputs/models/best_model.joblib

# Ensemble (recommended)
python -m src.predict_blind \
    --fasta blind_test.fasta \
    --model outputs/models/best_model.joblib \
    --model-finetune outputs/models/finetune_artifact.joblib

# Fine-tuned model only
python -m src.predict_blind \
    --fasta blind_test.fasta \
    --model outputs/models/finetune_artifact.joblib
```

---

class_distribution.pngsequence_length_distribution.pngcm_*.pngmodel_comparison.p
ablation_results.pngfeature_importance.pngshap_importance.pngper_class_metrics.p
reliability_diagram.pnghigh_confidence_errors.png

StandardScaler.fit()X[train_idx].transform()

42

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
```

```
    StratifiedKFold(shuffle=True, random_state=42)
    random_state=42
```

```
    src/features/new_feature.pycomposition.py
        extract_new_features(sequences: list[str]) -> np.ndarray
        get_feature_names() -> list[str]
    src/models/advanced.py
    src/predict_blind.py
```

```
    src/models/make_catboost()
    cross_validate_modelsrc/training.py
    src/evaluation.py
```

esm2_t33_650M_UR50D

```
    ESM_MODEL_NAMEEMB_DIMsrc/models/finetune.py
    MAX_LENbatch_size
```

```
    --batch-size
    --max-len 256
    --grad-accum 8
```

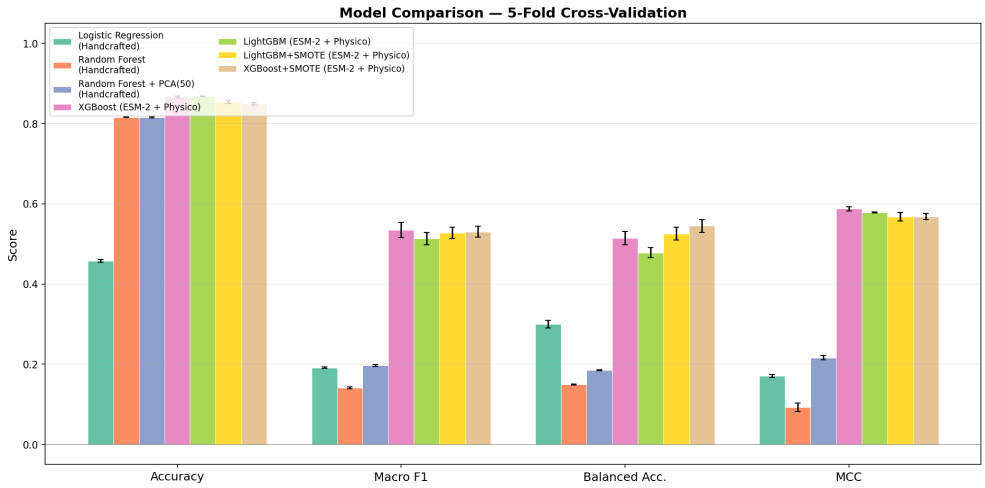
fair-esm

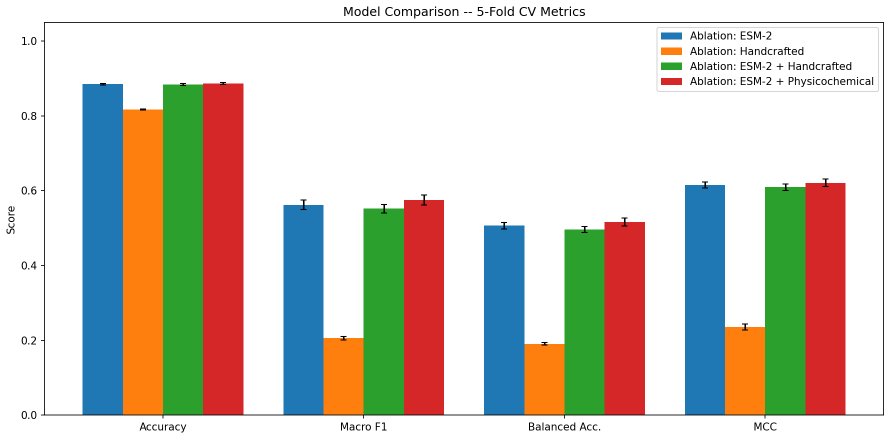
torch.hub

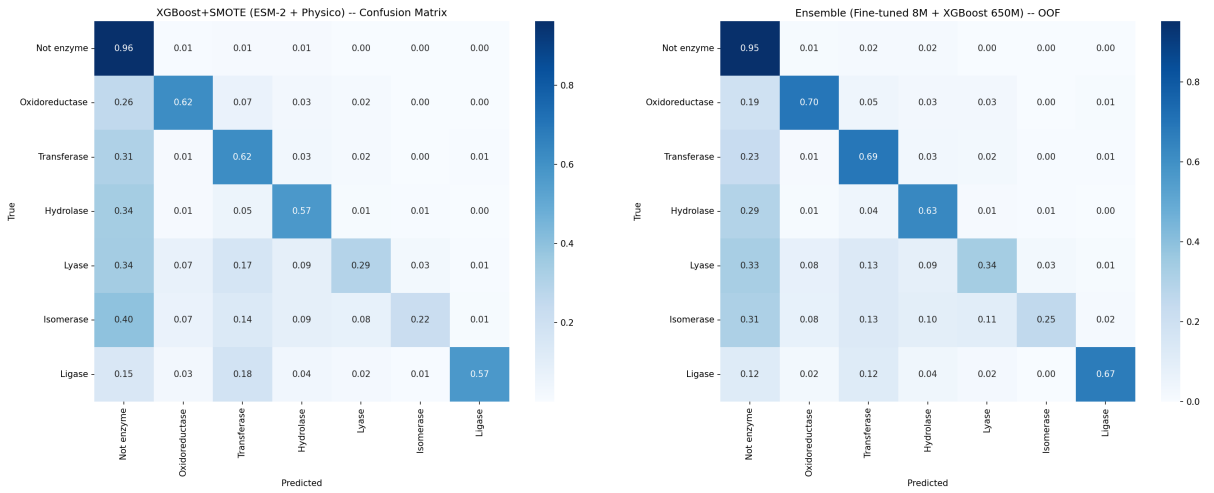
k_neighbors + 1k=5k_neighbors

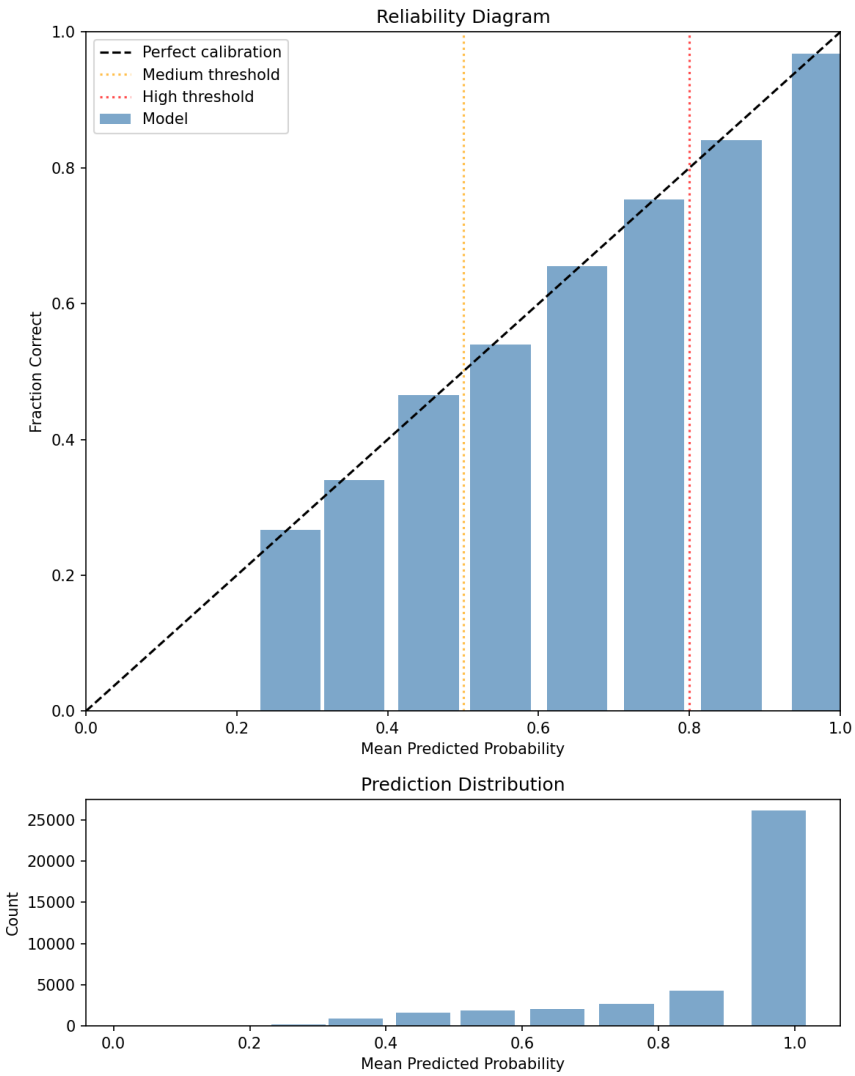
---

baseline_results.jsonadvanced_results.jsonfinetune_results.jsonensemble_results.confidence_results.jsoninterpretability_results.jsonbest_model.joblibfinetune_artifact.joblibfinetune_final.ptfinetune_fold{1-5}.pt






XGBoost+SMOTE (ESM-2 + Physico) -- Confusion Matrix
Not enzyme 0.96 0.01 0.01 0.01 0.00 0.00 0.00
Oxidoreductase 0.26 0.62 0.07 0.03 0.02 0.00 0.00
Transferase 0.31 0.01 0.62 0.03 0.02 0.00 0.01
Hydrolase 0.34 0.01 0.05 0.57 0.01 0.01 0.00
Lyase 0.34 0.07 0.17 0.09 0.29 0.03 0.01
Isomerase 0.40 0.07 0.14 0.09 0.08 0.22 0.01
Ligase 0.15 0.03 0.18 0.04 0.02 0.01 0.57
True
Not enzyme Oxidoreductase Transferase Hydrolase Lyase Isomerase Ligase
Predicted
0.8 0.6 0.4 0.2
Ensemble (Fine-tuned 8M + XGBoost 650M) -- OOF
Not enzyme 0.95 0.01 0.02 0.02 0.00 0.00 0.00
Oxidoreductase 0.19 0.70 0.05 0.03 0.03 0.00 0.01
Transferase 0.23 0.01 0.69 0.03 0.02 0.00 0.01
Hydrolase 0.29 0.01 0.04 0.63 0.01 0.01 0.00
Lyase 0.33 0.08 0.13 0.09 0.34 0.03 0.01
Isomerase 0.31 0.08 0.13 0.10 0.11 0.25 0.02
Ligase 0.12 0.02 0.12 0.04 0.02 0.00 0.67
True
Not enzyme Oxidoreductase Transferase Hydrolase Lyase Isomerase Ligase
Predicted
0.8 0.6 0.4 0.2 0.0


Reliability Diagram
Perfect calibration
Medium threshold
High threshold
Model
Fraction Correct
0.0 0.2 0.4 0.6 0.8 1.0
Mean Predicted Probability
Prediction Distribution
Count
25000 20000 15000 10000 5000 0
0.0 0.2 0.4 0.6 0.8 1.0
Mean Predicted Probability